PORTFOLIO ASSESSMENT 3 | OPTION 3D

Advanced ML and Deep Learning

Building heating-demand classification

Nguyen Van Tuan Kiet
Student ID: 104772099
COS40007 - Artificial Intelligence Engineering
Swinburne Vietnam | Class identifier: COS40007
Prepared: 2 November 2026

REFERENCE IMPLEMENTATION - AUTHORSHIP STATUS

OpenAI Codex substantially generated the code, experiment, analysis and report. The supplied brief explicitly prohibits whole-code GenAI generation. This reference is not evidence of independent student authorship, and disclosure alone does not satisfy that restriction. Student review and in-person demonstration remain pending.

Purpose and outcome

Continue the Portfolio 2 Energy Efficiency classification task and compare two traditional-model implementations with three Keras ANN batch-size variants on identical data. Report measured accuracy, class performance, probability loss and a geometry-disjoint stress check.

Training CV selects **Random Forest**; validation loss selects **ANN batch 16**. The observed leader on the reused 231-row comparison set is **Random Forest**. These results do not establish readiness for unseen real buildings.

Acknowledgement of Country

I acknowledge the Traditional Owners of the lands on which Swinburne operates and pay respect to Elders past and present. I recognise the continuing connections of Aboriginal and Torres Strait Islander peoples to Country, culture and community.

GitHub repository: github.com/odmiefnf/COS40007
Portfolio 3 code, notebook, models and results: portfolio/assessment-3
Canvas submission has not been performed.

A.1-A.2 | DATASET AND PREPARATION

A fixed, comparable task

The UCI Energy Efficiency dataset contains 768 simulated building configurations and eight explanatory variables [1]. It is the replacement dataset used for Portfolio 2. No claim is made that it was the dataset originally submitted in Portfolio 1.

Inputs	Handling	Reason
Compactness; surface, wall and roof areas; height; glazing area	StandardScaler	Six numerical features with different scales.
Orientation; glazing distribution	One-hot encoding	Nominal categories should not imply an ordinal distance.
Y1 heating load; Y2 cooling load	Excluded from predictors	Y1 defines labels only. Y2 would introduce an unavailable related outcome.

The fixed PA2 heating-load thresholds are **14.463333** and **28.406667** dataset units. Lower is at or below the first cutoff, Middle is above the first and at or below the second, and Higher is above the second. These are relative demand bands, not official efficiency ratings.

Partition	Rows	Lower	Middle	Higher	Encoded shape
fit	429	143	143	143	(429, 16)
validation	108	36	36	36	(108, 16)
test	231	67	76	88	(231, 16)

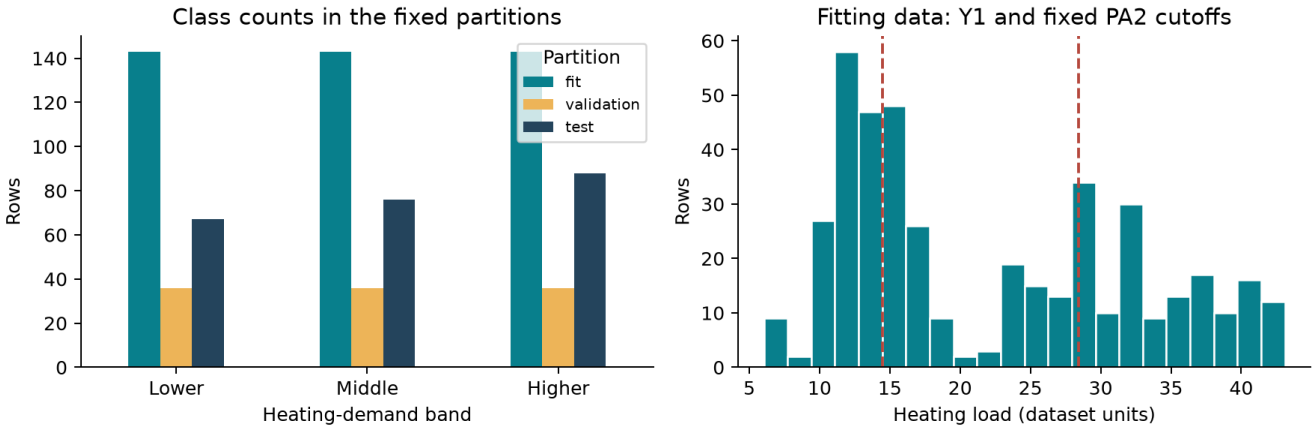


Figure 1. Actual class counts and fitting-only heating-load distribution. The raw source has no missing, nonfinite or duplicate rows.

Source CSV SHA256: 483904d33ab6db8bb0d1791d81410c18567c6b967035325a844de8ca97046ddc

A.3-A.5 | TRADITIONAL ML

Cross-validation and comparison

Preserve the PA2 537/231 development/test membership. Reserve 108 stratified development rows for validation and fit every main model on the remaining 429. Preprocessing is refitted inside each training CV fold. Both traditional models use four configurations and identical stratified five-fold splits, scored by accuracy and weighted F1.

Model	Selected configuration	CV accuracy ± SD	CV weighted F1
Random Forest	{'model__max_depth': None, 'model__min_samples_leaf': 1}	0.9184 ± 0.0129	0.9175
sklearn MLP	{'model__alpha': 0.01, 'model__hidden_layer_sizes': (64, 32)}	0.7879 ± 0.0538	0.7695

Random Forest uses 300 bootstrap trees with depth/leaf-size control. The sklearn MLP uses ReLU, Adam, L2 regularisation and an internal 10% early-stopping split. It remains a neural network; the report uses the assessment's model categories rather than claiming a fundamental distinction from ANN.

Model	Accuracy	Weighted F1	Macro F1	Crossentropy
Random Forest	93.07%	0.9300	0.9298	0.1876
sklearn MLP	82.68%	0.8222	0.8232	0.4279

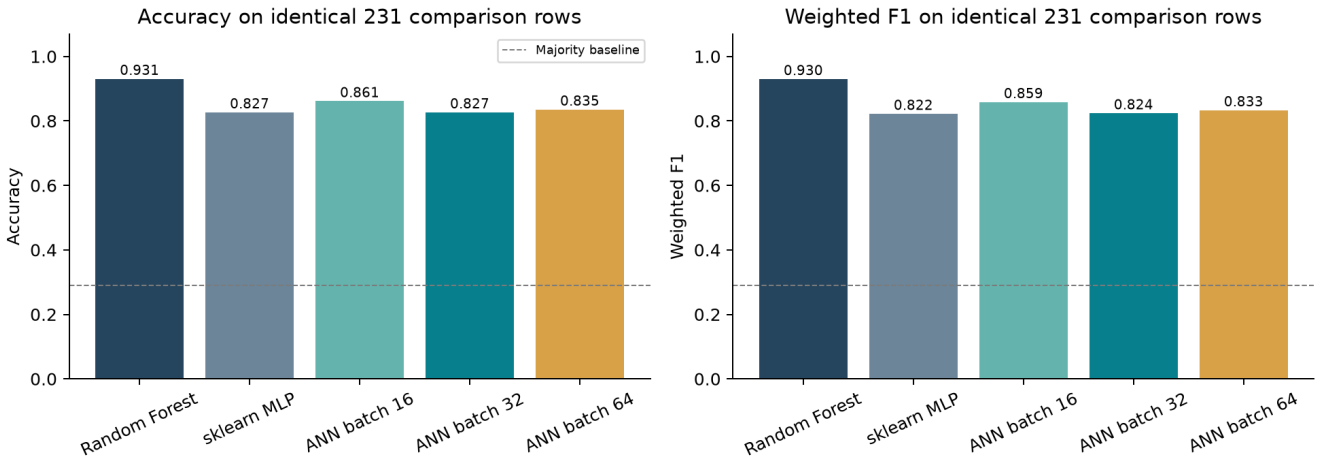


Figure 2. The required primary metric and weighted F1 across both traditional models and all three ANN variants. The majority-class baseline is shown only on accuracy.

Selection uses training CV. Grid-search CV is not nested and may be optimistic for the selected configuration. The 231 comparison rows were already examined in Portfolio 2; their reuse enables like-for-like comparison but not a new untouched external test.

B.1-B.2 | DEEP LEARNING

The specified ANN architecture

Energy data are non-sequential rows without image structure; option 3d is appropriate. The same 16 encoded input dimensions and the same 429/108/231 rows are used for all main models. Scaling is fitted only on 429 fitting rows. No sequence reshape is applied.

Layer	Output dimensions	Parameters	Purpose
Input	16	0	Eight source features become 16 encoded inputs.
Dense + ReLU	64	1,088	Learn nonlinear feature combinations.
Dropout (0.2)	64	0	Regularise the first hidden layer.
Dense + ReLU	32	2,080	Combine learned representations.
Dropout (0.2)	32	0	Regularise the second hidden layer.
Dense + ReLU	16	528	Compact hidden representation.
Dense + Softmax	3	51	Probabilities for Lower/Middle/Higher.
Total trainable parameters		3,747	No convolutional or recurrent layers.

Compilation uses Adam at learning rate 0.001, sparse categorical crossentropy and accuracy. Sparse loss matches integer labels 0/1/2. Softmax probabilities sum to one; dropout is disabled during inference. Keras 3 uses the installed PyTorch CPU backend. The model is saved as a genuine Keras .h5 file and as a native .keras checkpoint [5,6].

The complete executed model.summary() output is stored in the notebook and reproduced as a browser screenshot in the evidence appendix. Layer dimensions and parameter counts above are independently verified from reloaded .h5 artifacts.

B.3 | TRAINING AND MODEL SELECTION

Batch size is the controlled variable

Three experiments change batch size only: 16, 32 and 64. All start from seed 42 and share the prescribed architecture. Batch size affects the number of optimiser updates per epoch, so equal epochs do not imply equal update counts. Histories retain progress at epoch 10 and later milestones, with a ceiling of 50 epochs.

Setting	Implementation
Validation	Explicit 108-row stratified validation partition (20% of development).
Early stopping	Monitor val_loss; patience 10; restore best weights.
Learning-rate reduction	Monitor val_loss; halve rate after 5 plateau epochs; minimum 0.000001.
Checkpoint and history	Save minimum-val-loss .keras checkpoint; export .h5; pickle and CSV histories.
Selection	Lowest validation crossentropy; comparison-set accuracy is not used to choose batch size.

ANN variant	Epochs run	Best epoch	Validation loss	Validation accuracy
ANN batch 16	33	23	0.3220	86.11%
ANN batch 32	44	34	0.3368	86.11%
ANN batch 64	50	49	0.3432	86.11%

ANN batch 16 is the selected ANN. Each reported main model is trained only on the common fitting partition; no model receives extra validation rows for a final refit. This keeps the comparison controlled.

The script loads the exact minimum-validation-loss checkpoint before evaluation. It does not assume the last epoch is optimal. A learning-rate reduction occurs only if the monitored plateau condition is met; a configured callback is not evidence that it necessarily fired.

B.3 | OBSERVED LEARNING

Training and validation histories

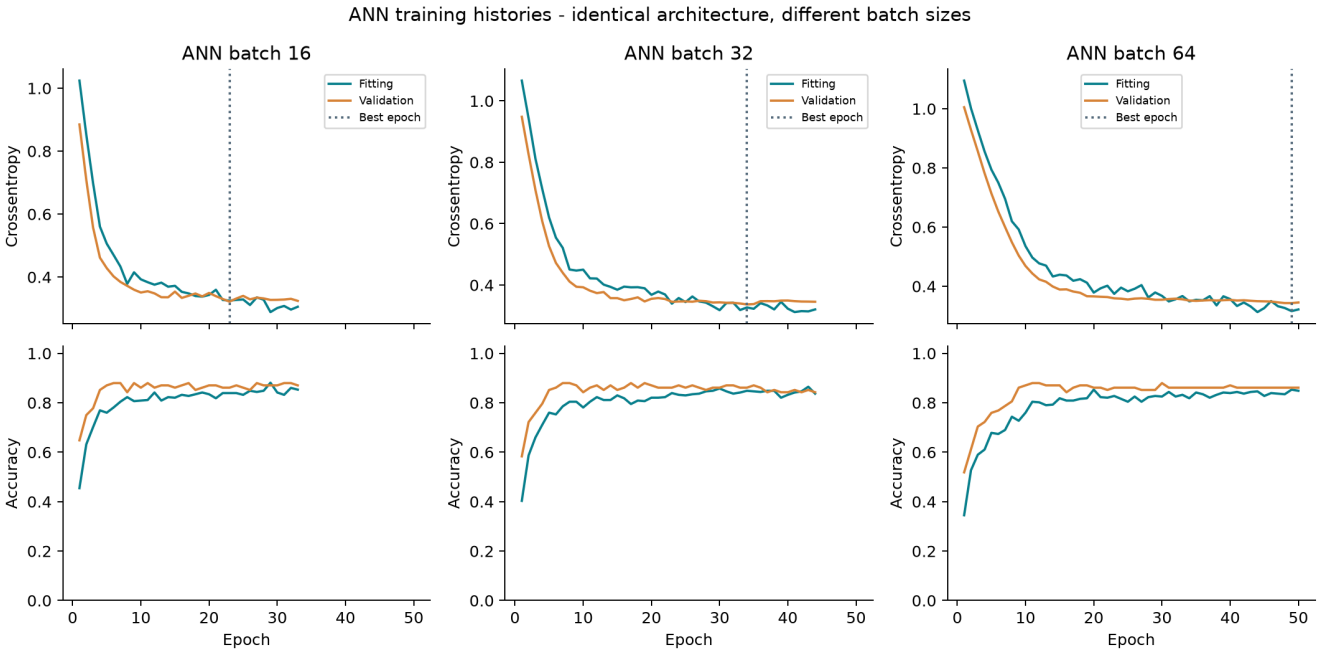


Figure 3. Actual fitting/validation crossentropy (top) and accuracy (bottom). The vertical marker identifies the minimum-validation-loss epoch. Every run is capped at 50 epochs.

ANN batch 16: validation loss changes from 0.8847 at epoch 1 to 0.3243 at the last epoch; the saved checkpoint is epoch 23. Validation accuracy at the selected checkpoint is 86.11%.

ANN batch 32: validation loss changes from 0.9481 at epoch 1 to 0.3455 at the last epoch; the saved checkpoint is epoch 34. Validation accuracy at the selected checkpoint is 86.11%.

ANN batch 64: validation loss changes from 1.0051 at epoch 1 to 0.3453 at the last epoch; the saved checkpoint is epoch 49. Validation accuracy at the selected checkpoint is 86.11%.

Loss and accuracy measure different properties. Crossentropy can worsen while hard-label accuracy stays constant when incorrect predictions become overconfident. Divergence between fitting and validation is an overfitting signal, but dropout makes training/evaluation curves intrinsically different. These data compare batch-size regimes; they do not isolate the causal benefit of dropout or depth.

B.4 | EVALUATION

Class-level behaviour

ANN variant	Accuracy	Weighted F1	Macro F1	Test loss
ANN batch 16	86.15%	0.8585	0.8593	0.3224
ANN batch 32	82.68%	0.8238	0.8244	0.3345
ANN batch 64	83.55%	0.8333	0.8340	0.3345

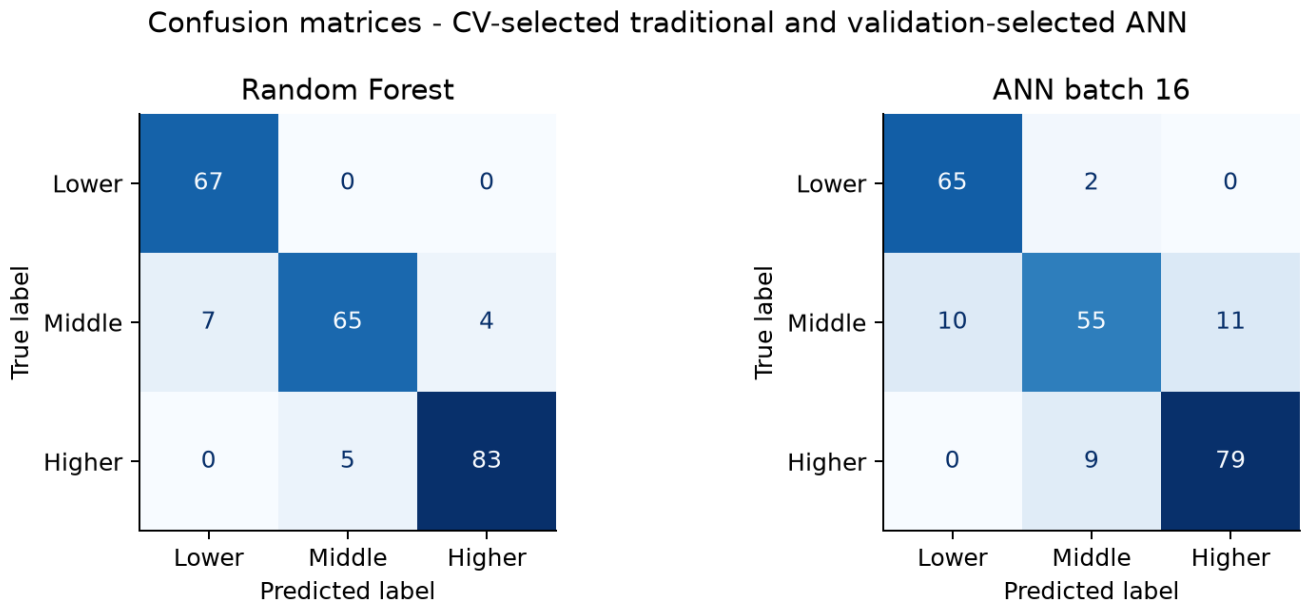


Figure 4. Predictions from the CV-selected traditional model and validation-selected ANN on the identical 231 reused comparison rows.

ANN batch 16	Precision	Recall	F1	Support
Lower	0.8667	0.9701	0.9155	67
Middle	0.8333	0.7237	0.7746	76
Higher	0.8778	0.8977	0.8876	88

The notebook includes full per-class reports for the selected traditional model and ANN. Class imbalance is modest, but weighted F1 alone can hide minority-class errors; macro F1 and the confusion matrix make the pattern visible. Boundaries between relative classes are sharp even where physical demand changes smoothly.

C.1-C.2 | MASTER COMPARISON

What the results support

Model	Accuracy	Weighted F1	Macro F1	Loss
Random Forest	93.07%	0.9300	0.9298	0.1876
sklearn MLP	82.68%	0.8222	0.8232	0.4279
ANN batch 16	86.15%	0.8585	0.8593	0.3224
ANN batch 32	82.68%	0.8238	0.8244	0.3345
ANN batch 64	83.55%	0.8333	0.8340	0.3345

Training CV selects **Random Forest**. Validation loss selects **ANN batch 16**. ANN minus selected traditional accuracy is **-6.93 percentage points**. The observed comparison-set leader is **Random Forest**; validation accuracy, with loss as a tie-breaker, selects **Random Forest** as the overall candidate.

The paired 2,000-resample row bootstrap interval for the ANN-minus-traditional accuracy difference is [-11.69, -2.16] percentage points. The exact paired McNemar p-value is 0.0090. These are descriptive calculations on reused, geometry-correlated rows; they cannot establish uncertainty for unseen building shapes.

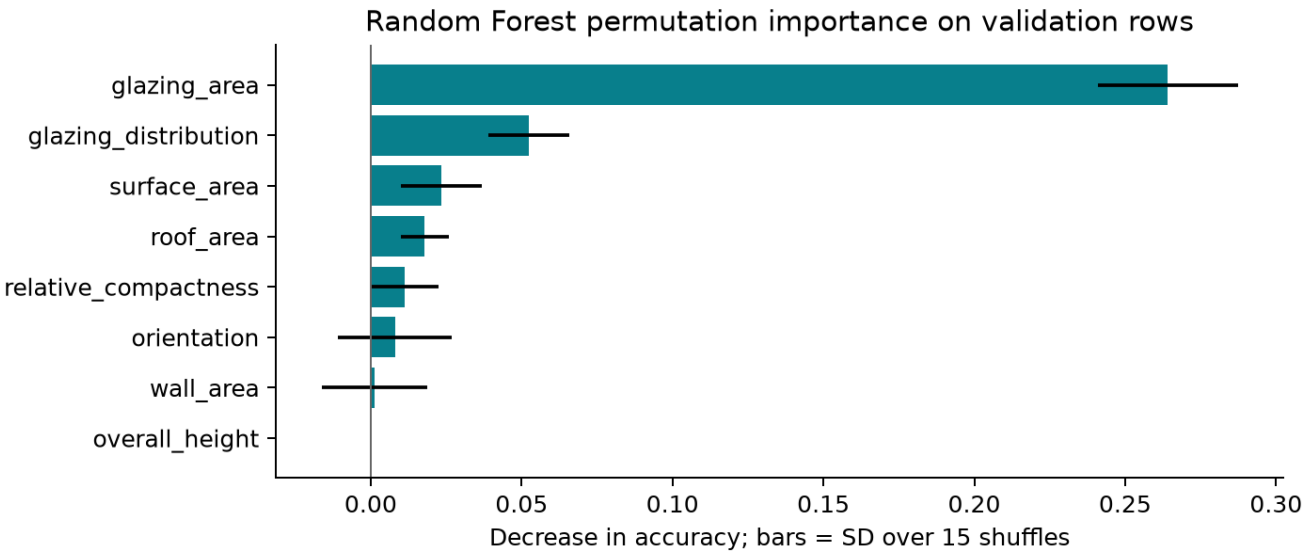


Figure 5. Random Forest validation permutation importance, with SD over 15 permutations. Correlated geometry predictors can substitute for one another; this is not a causal ranking.

ADDITIONAL ROBUSTNESS | DIFFERENT GENERALISATION QUESTION

Hold out whole geometries

The eight source variables describe only **12 underlying building geometries**. All **12** occur in both development and comparison partitions. Orientation/glazing variants of familiar shapes can therefore make a random split look stronger than prediction for unseen shapes.

Use three GroupKFold outer folds across the full dataset. In each outer training partition, reserve 20% of remaining geometry groups for validation; fit preprocessing and all models only on the remaining groups. Fixed RF (300 trees, unrestricted depth, leaf=1), fixed MLP (64/32, alpha=.01) and fixed ANN (batch 32) avoid importing the row-CV-selected settings. No grouped tuning is claimed.

Fixed model	Accuracy mean	Accuracy SD	Weighted F1 mean
ANN batch 32	69.27%	0.0642	0.6892
Random Forest	64.06%	0.1004	0.6426
sklearn MLP	67.84%	0.0623	0.6756

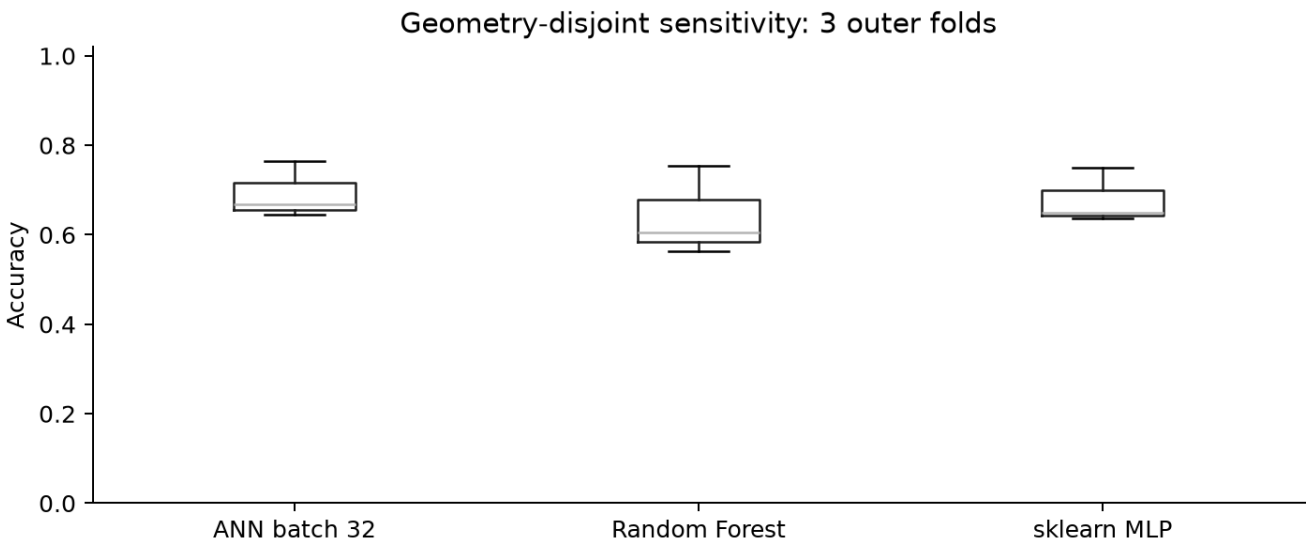


Figure 6. Three accuracy values per fixed model, each withholding complete geometries. Small fold count and the fixed PA2 label definition limit interpretation.

Do not deploy yet. This simulation-only evidence lacks independent real-building validation, probability calibration, drift checks and operational cost assessment. A building-level external sample and a clear classification-versus-regression objective are needed before practical use.

C.3 | REFLECTION QUESTIONS

Interpretation and next experiments

How did architecture choices affect performance?

Dense 64/32/16 layers learn nonlinear feature combinations; three softmax outputs match the target classes. Dropout 0.2 regularises the first two hidden layers. Batch size changes gradient noise and update frequency; validation histories provide direct evidence for that comparison. There was no width/depth or no-dropout ablation, so the results cannot quantify how much dropout or additional layers caused any improvement.

Did deep learning outperform traditional ML? Why?

The selected ANN scores 86.15% accuracy versus 93.07% for the selected traditional model on reused rows, a difference of -6.93 percentage points. The master table adds weighted/macro F1 and probability loss. A Random Forest can capture threshold interactions efficiently in small tabular data, while an ANN depends on optimisation and regularisation. These are plausible mechanisms, not isolated causal findings. The sklearn MLP is also a neural network. The dataset and validation design do not justify a universal claim of deep-learning superiority.

With more time, what would improve performance?

Collect independent real-building observations and evaluate complete unseen geometries. Use nested grouped cross-validation for model selection and repeat seeds. Test dropout and architecture ablations, probability calibration, ordinal penalties and sensitivity near the fixed cutoffs. Evaluate continuous heating-load regression if that better serves the practical objective. Any physical feature engineering should be fitted and assessed within the validation design, with Y1/Y2 kept out of predictors.

Would the best model be deployed in production?

No. Strong scores for familiar geometry do not establish performance for new building shapes or climates. The classes are dataset-relative rather than regulatory labels, and the 231 rows have been reused from an earlier assessment. External building-level validation, calibrated probabilities and defined error costs are required first.

These are assistant-drafted technical reflections. They do not certify that the student personally derived the conclusions or completed the required in-person studio demonstration.

E | MANDATORY REPORT-ONLY TEMPLATE

GenAI Reflection

1. GenAI Tools Used

OpenAI Codex (GPT-6-based agent in the Codex desktop app; exact served build not exposed): retrieved the brief; inspected prior work; designed and generated the full notebook and supporting code; executed models; checked artifacts; drafted the interpretation, report and demo guide. No other generative AI tool is claimed. Python, scikit-learn, Keras, PyTorch and ReportLab were execution libraries.

2. Specific Tasks & Prompts

Task 1: access Canvas and complete Portfolio 3. Exact user prompt:

"truy cập vào cavas swindburne của tôi, tìm kiếm assignment Portfolio Assessment 3: Advanced ML - Regression/Classification using ML and Deep Learning rồi hoàn thiện bài tập"

Task 2: locate the exact assignment. Exact URL supplied:

"https://swinburne.instructure.com/courses/78034/assignments/803620"

Task 3: publish to the same GitHub repository and add links to the work. Exact user prompt:

"đẩy lên github của tôi và gắn link vào bài làm như bài trươcs"

The user replied "ok" to using COS40007 temporarily as the class identifier. No separate student-written debugging prompts are invented. The full record is in GENAI_EVIDENCE.md.

3. Code Attribution only for debugging and helping find errors

The actual use exceeds that category. All new notebook modelling sections, experiment design, verification/inference/rendering scripts and drafted interpretations are substantially AI generated. Describing them as only debugging assistance would be inaccurate. The brief explicitly prohibits whole-code generation; this package is therefore labelled a reference. Disclosure alone does not remove that restriction or demonstrate independent student authorship.

4. Learning Reflection

How could GenAI help learning? The executed example makes fold-specific scaling, validation-based checkpointing and model reload checks inspectable. It can support a student-led explanation and reconstruction of the workflow.

What challenges did it create? Full generated code exceeds the brief's allowed scope. The distinction between reusable row comparison and unseen-geometry validation also needs careful review. A Windows library-loading issue required compatible library versions; no security setting was changed.

What specific skills were developed? Student skill development has not been assessed. Relevant skills to demonstrate are target definition, leakage control, five-fold CV, interpreting softmax/crossentropy, reading learning curves and reloading model artifacts. No personal learning experience, independent authorship or completed student verification is asserted.

B.5 AND D | REPRODUCIBILITY AND REPOSITORY

Artifacts that can be checked

Artifact	Content / verification
One Jupyter notebook	Full Parts A-C, D, references, executed code and actual stored outputs.
random_forest.pkl; sklearn_mlp.pkl	Fitted preprocessing plus estimator; probabilities checked against exported rows.
ann_batch16/32/64.h5 and .keras	Complete ANN variants; layer units, dropout rates and 3,747 parameters verified on reload.
ann_preprocessor.pkl; metadata JSON	Fitted encoder/scaler, source feature order, class order, fixed thresholds and versions.
ANN history .pkl / epoch CSVs	Training/validation losses and accuracy; epoch count and selected checkpoint.
Metrics, predictions and figures	Comparison tables and probability-level evidence; independent metric recalculation.

Exact execution environment

Component	Version
python	3.11.9
numpy	2.4.6
pandas	2.3.3
scikit-learn	1.7.2
keras	3.12.0
torch	2.14.1+cu130
backend	torch
device	CPU

Run: python -m pip install -r requirements.txt; then Restart Kernel and Run All from the notebook folder. The data are included. tools/predict.py demonstrates inference on a CSV containing X1-X8. tools/verify_analysis.py independently recalculates metrics and checks stored model predictions and executed cells.

GitHub development record

Code, the executed notebook, saved models, results and reproducibility files are available in portfolio/assessment-3 on the main branch of odmiefnf/COS40007, the same repository as Portfolio 1 and 2. Real local development milestone commits are retained in the integration history and attributed to OpenAI Codex. Publication was separately requested by the student; it does not establish independent student authorship. No history was backdated or invented.

No Canvas submission has been performed. Canvas displayed a 1 November 23:59 due date and 5 November 23:59 availability cutoff when accessed on 2 November 2026. The printed May date in the option PDF is stale relative to the live assignment; displayed times use the Canvas account setting.

NOTEBOOK EVIDENCE | B.2

Executed architecture summary

This screenshot captures the rendered output of the executed notebook cell. The full uncropped browser view, including source code, is also retained in output/evidence_architecture_full.png. It is shown separately from the structured table so that the original model output remains legible.

Stored notebook output

```
Model: "heating_demand_ann"
```

Layer (type)	Output Shape	Param #
dense_64 (Dense)	(None, 64)	1,088
dropout_1 (Dropout)	(None, 64)	0
dense_32 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_16 (Dense)	(None, 16)	528
demand_band (Dense)	(None, 3)	51

```
Total params: 3,747 (14.64 KB)
```

```
Trainable params: 3,747 (14.64 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

Rendered directly from saved executed notebook outputs. No values entered manually.

Screenshot 1. Actual saved notebook output viewed in a local browser. The screenshot has been cropped to the summary, with no changes to values.

The notebook shows the exact input shape, dropout and compilation code. Reload verification checks dense units [64, 32, 16, 3], dropout rates [0.2, 0.2] and 3,747 trainable parameters. The parameter count matches the manually explained architecture.

NOTEBOOK EVIDENCE | A.4 AND B.3

Executed comparisons and checkpoints

Browser views of the saved notebook outputs; the comparison view displays the primary metric columns. Complete tables and raw values remain in the notebook and CSV files.

COS40007 | PORTFOLIO 3 | AI-GENERATED REFERENCE

A.4 / B.4 - Executed comparison results

Nguyen Van Tuan Kiet | 104772099 | Cell execution 9

Stored notebook output

```
Validation-selected overall candidate: Random Forest
Observed comparison-set leader (descriptive): Random Forest
Majority-class comparison-set baseline: 0.29
```

model	type	test_accuracy	test_weighted_f1	test_macro_f1	test_loss
Random Forest	Traditional ML	0.9307	0.9300	0.9298	0.1876
sklearn MLP	Traditional ML	0.8268	0.8222	0.8232	0.4279
ANN batch 16	Deep Learning	0.8615	0.8585	0.8593	0.3224
ANN batch 32	Deep Learning	0.8268	0.8238	0.8244	0.3345
ANN batch 64	Deep Learning	0.8355	0.8333	0.8340	0.3345

Rendered directly from saved executed notebook outputs. No values entered manually.

Screenshot 2. Actual executed comparison table and selection outputs; displayed columns are a subset of the complete stored table.

COS40007 | PORTFOLIO 3 | AI-GENERATED REFERENCE

B.3 - Executed training and checkpoint selection

Nguyen Van Tuan Kiet | 104772099 | Cell execution 7

Stored notebook output

model	batch_size	epochs_run	best_epoch	validation_loss	validation_accuracy
ANN batch 16	16	33	23	0.3220	0.8611
ANN batch 32	32	44	34	0.3368	0.8611
ANN batch 64	64	50	49	0.3432	0.8611

Rendered directly from saved executed notebook outputs. No values entered manually.

Screenshot 3. Actual epoch counts, checkpoint epochs and validation results for all three batch sizes.

SOURCES AND TRACEABILITY

References and rubric map

[1] Tsanas, A. and Xifara, A. (2012). Energy Efficiency [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C51307. CC BY 4.0. https://doi.org/10.24432/C51307

[2] Swinburne Canvas. Portfolio Assessment 3: Advanced ML - Regression/Classification using ML and Deep Learning. Course 78034, assignment 803620; option 3d PDF. Accessed 2 November 2026. https://swinburne.instructure.com/courses/78034/assignments/803620

[3] Scikit-learn. RandomForestClassifier API. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[4] Scikit-learn. Neural network models (supervised). https://scikit-learn.org/stable/modules/neural_networks_supervised.html

[5] Keras. Training and evaluation with built-in methods. https://keras.io/guides/training_with_built_in_methods/

[6] Keras. Callbacks API and ModelCheckpoint. https://keras.io/api/callbacks/ ; https://keras.io/api/callbacks/model_checkpoint/

Rubric / brief component	Evidence
Traditional models and visualisation	A.3-A.5; two pipelines, five-fold CV, master table and Figure 2.
Deep-learning preparation	A.1-A.2 and B.1; source hash, exact rows, class counts, shape and fitted-only scaling.
Architecture and training	B.2-B.3; exact 64/32/16 network, callbacks, 3 batch sizes, actual histories.
Deep-learning evaluation	B.4; accuracy/loss, per-class report, confusion matrix and Figure 3 curves.
Final summary / reflection	C.1-C.3; selected versus observed winners, limits and three reflection questions.
Model saving	B.5; .h5, .pkl and native checkpoints; independent reload verification.
GitHub integration	Direct link on the cover; complete code, models, results and retained milestone history.
GenAI reflection	E; exact four-part template with truthful full-assistance disclosure.
Studio demo	Student-led in-person performance remains outstanding (20 rubric points).

The report explains actual computed artifacts; it does not certify compliance with the independent-authorship restriction. Read GENAI_EVIDENCE.md and the Vietnamese demo guide before deciding how to use the reference.